ISL Recognition System - Project Overview and Change Log

Prepared as a plain-text record of the work carried out on this codebase.

1. What this project is

The ISL Recognition System is a single Flask application that recognizes Indian Sign Language (ISL) hand signs in real time from a webcam feed and converts them into text and speech. It has three main parts working together:

- A camera and computer vision pipeline (isl_main.py) that captures video, uses Google MediaPipe to detect hand landmarks (21 points per hand), smooths those landmarks with a One-Euro filter, and feeds them into a machine learning classifier (a scikit-learn Random Forest) trained to recognize the A-Z alphabet plus a set of whole-word gestures (HELLO, LOVE, THANK YOU, HELP, SORRY, YES, NO, GOOD, PLEASE).

- A Trainer tab that lets someone record new hand-sign samples through the browser, store them in isl_dataset.csv, and retrain the live model without restarting the server.

- A plain HTML/CSS/JavaScript frontend (templates/index.html, static/css/style.css, static/js/app.js) served directly by the same Flask process, showing the live camera feed with hand-tracking overlay, the recognized letters/words, a reference guide to every sign, and (for admins) the training tools.

There is also a small collection of supporting scripts and documents: dataset utilities (fix_dataset.py, process_images.py, import_kaggle_dataset.py), model evaluation and reproducibility reporting (evaluate.py, docs/reproducibility_checklist.md, docs/data_collection_protocol.md), and an academic writeup (paper/main.tex).

2. Starting point of this engagement

Work in this engagement began partway through an existing task: a login and role-based access system had already been mostly written into isl_main.py, templates/login.html, static/img/, static/css/style.css, static/js/app.js, and templates/index.html, but the new files had not yet been added to git and the feature had not been verified end to end. The first step was to read through all of that in-progress work, confirm the Flask application still started correctly, and confirm the login page actually redirected and rendered as intended.

3. Login and role-based access system

Two fixed accounts were introduced: 'admin' and 'customer'. On the very first run of the application, random passwords are generated for both accounts, hashed with werkzeug's password hashing, and saved to a local, git-ignored file (auth_users.json). The plaintext passwords are printed to the console exactly once, at that first run, and are never stored anywhere in plaintext afterward.

The admin account has access to everything: live recognition, the Trainer tab, recording new samples, deleting or clearing samples, triggering model retraining, and changing the camera resolution. The customer account is restricted to live camera recognition and the hand-sign

reference guide only; the Trainer tab and 'How It Works' page are hidden from the interface and also blocked at the server if a customer session tries to call those endpoints directly.

A new login page (templates/login.html) and a logout button on the main page were added. Every page and API route now checks whether a valid session exists, and most training-related routes additionally check that the session belongs to the admin role.

4. Security hardening added on top of the login system

Once the application started relying on session cookies for access control, several supporting protections were added:

- CSRF protection: every POST request (the login form, the logout button, and every action the frontend sends while recording, training, or using the app) must now include a security token tied to that browser session. The token is embedded in the page and automatically attached to every request by the frontend code; requests missing or presenting the wrong token are rejected.

- Login rate limiting: if the same network address fails to log in five times within five minutes, further attempts from that address are blocked for five minutes. This slows down anyone trying to guess a password.

- Session cookie settings were tightened (marked HttpOnly and SameSite, with a defined 12-hour session length) to reduce the ways a session cookie could be misused.

- A small internal inconsistency was fixed in the background detection loop: the code that reads the current machine-learning model while recognizing a sign now reads it under the same protective lock that the training code uses when it replaces the model after retraining, removing a theoretical race condition between the two.

All of the above was tested directly: a temporary local server was started, logins were performed as both admin and customer, protected pages and actions were confirmed to require the correct session and the correct security token, and the rate limit was confirmed to trigger after five failed attempts.

5. Preparing the project to be published

The project was reviewed as a whole to identify anything that should be fixed before sharing or publishing the code. The following issues were found and corrected:

- The repository was carrying about 90 megabytes of experimental dataset and model files (Kaggle import results and a one-hand variant) that are not used by the running application at all. These were removed from version control (they remain on disk locally and can be regenerated with the existing import_kaggle_dataset.py script) and added to .gitignore so they are not tracked again by accident.

- No license file existed. An all-rights-reserved LICENSE file was added, matching the project owner's choice to keep the code private/proprietary rather than open source.

- The project's README.md did not mention the new login system at all. It was updated to explain how to retrieve the first-run passwords, how a password can be reset, which API routes require

an admin session, and the new security-token requirement on every POST request.

- A hardcoded description of the original developer's specific computer processor was found inside evaluate.py (the script that produces the project's accuracy/performance report). This was replaced with a call that detects the actual processor of whichever machine the script is run on, so the report is correct regardless of who runs it.

- A leftover, never-used piece of dead code inside evaluate.py was removed.

- fix_dataset.py, a command-line tool for editing the training dataset, previously deleted or trimmed recorded samples immediately upon request with no confirmation step. A confirmation prompt (type YES to proceed) was added before any of its three destructive operations run, so a mistaken keystroke cannot silently erase recorded training data.

- Two minor code-consistency issues inside isl_main.py were cleaned up: a couple of places were hardcoding the dataset filename directly instead of reusing the constant already defined for it, and two file-reading spots were missing a small option (newline handling) that the rest of the dataset code already used consistently.

- A stray, incorrectly named leftover image file was found among the newly added reference pictures and deleted.

During review, it was also noted that a couple of the 26 new reference hand-sign photographs (specifically the pictures shown for the letters A and O) do not clearly match the hand shape described in their own caption text. This was flagged directly; the project owner chose to keep the photographs as they are for now and review/replace them personally later, so no changes were made to the images themselves.

6. Verification performed

Before committing, the following was checked and confirmed working:

- Every changed Python file compiles without errors.
- The application starts successfully with the camera and hand-tracking model loading normally.
- Visiting the site without logging in correctly redirects to the login page.
- Logging in as admin grants access to the Trainer tab and its API routes.
- Logging in as customer is correctly denied access to those same admin-only routes (receives an access-denied response).
- A request that omits the required security token is correctly rejected, and a request that includes the correct token succeeds.
- Six rapid failed login attempts from the same address correctly trigger the temporary lockout on the sixth attempt.

7. Committing and publishing the work

All of the above changes were committed to the 'frontend' branch of the project's git repository in a single commit titled 'Add login/role auth (admin+customer), CSRF/rate-limit hardening, and publish cleanup', and then pushed to the project's remote repository on GitHub so the changes are now available there as well.

Note on repository size: removing the unused Kaggle/one-hand files from tracking stops the repository from growing further because of them, but it does not shrink the existing repository history, since those files still exist inside earlier, already-published commits. Actually shrinking the repository would require rewriting git history, which was intentionally not done because it would affect anyone who already has a copy of the repository; the project owner can request that separately if it becomes worth doing.

8. Login credentials note

For security, the actual admin and customer usernames and passwords are not written into this document. They were provided directly in the chat conversation instead, since this PDF will live inside the project folder and could end up being shared, screenshotted, or committed to version control. If those credentials are lost, they can be regenerated at any time by deleting auth_users.json in the project folder and restarting the application, which will print a brand-new set of passwords to the console on that next startup.